Cybersecurity

FedSHE：基于自适应分段CKKS同态加密的隐私保护和高效联邦学习

RESEARCH Open Access

FedSHE: privacy preserving and efficient federated learning with adaptive segmented CKKS homomorphic encryption

Yao Pan[1,2], Zheng Chao[1*], Wang He[1], Yang Jing[1], Li Hongjia[1] and Wang Liming[1]

Abstract

Unprotected gradient exchange in federated learning (FL) systems may lead to gradient leakage-related attacks. CKKS is a promising approximate homomorphic encryption scheme to protect gradients, owing to its unique capability of performing operations directly on ciphertexts. However, configuring CKKS security parameters involves a trade-off between correctness, efficiency, and security. An evaluation gap exists regarding how these parameters impact computational performance. Additionally, the maximum vector length that CKKS can once encrypt, recommended by Homomorphic Encryption Standardization, is 16384, hampers its widespread adoption in FL when encrypting layers with numerous neurons. To protect gradients' privacy in FL systems while maintaining practical performance, we comprehensively analyze the influence of security parameters such as polynomial modulus degree and coefficient modulus on homomorphic operations. Derived from our evaluation findings, we provide a method for selecting the optimal multiplication depth while meeting operational requirements. Then, we introduce an adaptive segmented encryption method tailored for CKKS, circumventing its encryption length constraint and enhancing its processing ability to encrypt neural network models. Finally, we present *FedSHE*, a privacy-preserving and efficient *Fed*erated learning scheme with adaptive *S*egmented CKKS *H*omomorphic *E*ncryption. *FedSHE* is implemented on top of the federated averaging (FedAvg) algorithm and is available at https://github.com/yooopan/FedSHE. Our evaluation results affirm the correctness and effectiveness of our proposed method, demonstrating that FedSHE outperforms existing homomorphic encryption-based federated learning research efforts in terms of model accuracy, computational efficiency, communication cost, and security level.

Keywords Federated learning, Gradient leakage, Homomorphic encryption, Segmented CKKS, FedAvg

Introduction

In recent years, with under-regulated industries and the ever-growing demand for stringent privacy requirements, federated learning (FL) Yang et al. (2019) has emerged as a critical solution to data silos and security issues. Federated learning (FL) McMahan et al. (2017) is a pioneering privacy-preserving distributed machine learning paradigm that allows multiple devices to collaboratively train a shared model while keeping the training data decentralized, i.e., the data is kept locally rather than sent to a central server. This approach is advantageous when the data is sensitive or cannot be easily moved due to privacy concerns or regulatory oversight.

According to the characteristics of data partition situations, Yang et al. (2019) categorized FL into horizontal federated learning (HFL), vertical federated learning (VFL), and federated transfer learning (FTL). The essence of HFL is the federation of samples, applying to scenarios when multiple devices possess the same features but

*Correspondence:
Zheng Chao
zhengchao1135@iie.ac.cn
[1] Institute of Information Engineering, Chinese Academy of Sciences, No. 19, Shucun Road, Haidian District, Beijing, China
[2] School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

Springer Open

© The Author(s) 2024. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

have different data samples. For instance, in intelligent voice assistants, users may articulate identical sentences (within the feature space) while exhibiting diverse types of speech (within the sample space). VFL involves multiple devices with different features but with a standard set of data samples. For instance, a financial institution and a credit rating agency might collaborate to create a credit scoring model, with the former providing customer data and the latter providing credit history data. FTL is suitable for scenarios where both feature and sample overlap are small, and the primary purpose is to cope with the problem of insufficient data and small data volume while ensuring the security and privacy of each device.

Federated learning takes a step towards protecting private data by exchanging gradients instead of raw data. For a long time, it was considered secure to exchange gradients due to not accessing the local data of the devices. Nonetheless, a series of studies has demonstrated that the publicly shared gradients can reveal private training data, either to an honest but curious central server or to a malicious adversary. Aono et al. (2017) observed that during neural network training, a tiny portion of the gradients might leak valuable information from the training data. Zhu et al. (2019) proposed an attack method named DLG (deep leakage from gradients) that uses gradients to reconstruct training data and verified the effectiveness of the attack on computer vision and natural language processing tasks. Zhao et al. (2020) improved DLG and proposed an improved algorithm iDLG. Geiping et al. (2020) changed the loss function of the attack algorithm to cosine similarity, further improving the attack accuracy. Dimitrov et al. (2022) propose a new optimization-based attack that successfully attacks the FedAvg algorithm and shows that many real-world FL implementations based on FedAvg are vulnerable. Wei et al. (2020) provided a framework for evaluating gradient leakage attacks in FL. These studies furnish ample evidence demonstrating the vulnerability of federated learning systems and underscore the imperative of adopting privacy-preserving mechanisms.

Utilizing encryption for masking before gradient aggregation proves to be an effective approach to ensuring security. Traditional encryption schemes such as AES and DES necessitate decryption of ciphertext before performing computations, rendering them unsuitable for FL. Homomorphic encryption (HE) is a powerful cryptographic primitive that allows computations on encrypted data so that only the secret key holder can decrypt the computation result, satisfying the privacy protection requirement of gradients. HE is characterized by its adaptability and high-level security, making it a preferred choice for implementing privacy-preserving FL. The additive HE scheme Paillier is widely employed in federated learning but entails high computational and communication costs. The CKKS (Cheon-Kim-Kim-Song) scheme Cheon et al. (2017, 2018, 2019) that benefits from ciphertext packing and rescaling has been validated as the most efficient HE scheme to perform approximate homomorphic computations over real and complex numbers. Although there have been several research efforts Stripelis et al. (2021); Qiu et al. (2022); Yao et al. (2023) attempting to utilize CKKS to enhance the security of FL systems, the following challenges remain to be overcome:

- *Complexity of security parameter selection.* The security parameters in the CKKS scheme are rather complex and involve a delicate balance between security, correctness, and efficiency. Setting security parameters correctly and efficiently is challenging for FL algorithm designers, which often needs to be clarified. The studies above overlooked parameter optimization when employing CKKS. It is imperative to systematically assess the correlation between security parameters and computational performance, followed by providing a straightforward method for parameter selection.
- *Limitation of encryption length.* Deep neural networks usually have numerous parameters. For example, AlexNet Krizhevsky et al. (2017) has 60 million neurons. Even the comparatively simple LeNet-5 LeCun et al. (1998) has more than 60,000 neurons. However, the maximum vector length that CKKS can once encrypt recommended by Homomorphic Encryption Standardization Albrecht et al. (2021) is 16384, which results in insufficient capacity when encrypting layers with a large number of neurons, hindering the widespread adoption of CKKS in the field of privacy-preserving FL.

To utilize homomorphic encryption to defend against gradient leakage attacks while maintaining practical performance, this paper makes the following contributions:

1. We first analyze the relationship between security parameters such as polynomial modulus degree and coefficient modulus and evaluate the impact of security level and multiplication depth on the performance of homomorphic operations. Derived from our evaluation findings, we provide a methodology for selecting the optimal multiplication depth while meeting the prerequisites of homomorphic operations.
2. We propose an adaptive segmented encryption method for CKKS based on the evaluation conclusions. This method effectively circumvents the

encryption length constraint of CKKS, enhancing its processing ability to encrypt neural network models with numerous neurons.

3. We present **FedSHE**, a privacy preserving and efficient **Fed**erated learning scheme with adaptive **S**egmented CKKS **H**omomorphic **E**ncryption. We implement FedSHE on top of the federated averaging (FedAvg) algorithm with optimal CKKS parameters and evaluate on multiple datasets. The source code is available at https://github.com/yooopan/FedSHE. Our evaluation results demonstrate the correctness and effectiveness of our proposed method and indicate that FedSHE outperforms existing homomorphic encryption-based federated learning research efforts in terms of model accuracy, computational efficiency, communication cost, and security level.

The subsequent sections of this paper are structured as follows: Section "Preliminaries" provides an overview of background knowledge, including FedAvg, gradient leakage attacks, and homomorphic encryption. Section "Related work" outlines relevant research endeavors. Section "Segmented CKKS" introduces the segmented CKKS encryption algorithm proposed in this study. Section "Proposed FedSHE scheme" presents the FedSHE scheme proposed in our research. Section "Performance evaluation and analysis" encompasses implementation, evaluation, and analysis. Conclusions are ultimately presented in section "Conclusions".

Preliminaries

In this section, we will introduce the training process of the FedAvg algorithm, explain why the original FedAvg is vulnerable to privacy attacks, and discuss why CKKS is preferred among various HE schemes.

Federated averaging algorithm

FedAvg is a standard horizontal federated learning training method with client–server architecture. Key steps of FedAvg are further explained as follows.

- *Initialization*. In FedAvg, the primary objective is to find a global model W that minimizes a global loss function by aggregating local updates from individual clients. The global objective function in FedAvg can be defined as the average of local loss functions across all clients: $\min_W \mathcal{L}(W) = \frac{1}{K}\sum_{k=1}^{K}\mathcal{L}_k(W)$, where K is the total number of clients, $\mathcal{L}_k(W)$ represents the local loss function for client k, and W is typically created with random parameters.
- *Client Updates*. In each communication round t, client k performs a local model update by minimizing its local loss function $\mathcal{L}_k(W)$ using stochastic gradient descent (SGD) or other optimization method: $\Delta W_k^t = -\eta\nabla\mathcal{L}_k(W)$, where ΔW_k^t is the update to the local model, η is the learning rate, and $\nabla\mathcal{L}_k(W)$ is the gradient of the local loss concerning the model parameters.
- *Model Aggregation*. After local updates are computed, the central server aggregates these updates to obtain a new global model: $W^{t+1} = \frac{1}{K}\sum_{k=1}^{K}\Delta W_k^t$, where W^{t+1} is the global model for the next round, and ΔW_k^t is the update from client k in round t.

Client local training, server aggregation, and global model updates are repeated for multiple communication rounds until the global model converges.

Gradient leakage attack

Transmitting the gradients in plaintext may allow potential adversaries to launch gradient leakage-related attacks. Zhu et al. (2019) proposed a DLG (deep leakage from gradients) attack to reconstruct training data using gradients. Specifically, the adversary generates a pair of random dummy samples and labels (x', y') and then trains the model F. After obtaining the corresponding virtual gradients $\nabla W'$, the adversary optimizes the virtual input and label to minimize the distance between $\nabla W'$ and the real gradients ∇W. When the distance between $\nabla W'$ and ∇W is small enough, (x', y') and the original data (x, y) will highly match. Zhao et al. (2020) observed that DLG often generates wrong labels when reconstructing data and proposed improved DLG. They use the relationship between the probability of each label in the output layer and the gradients of the output value of the previous layer to find out the real label y accurately. When solving the optimization problem: $x'^*, y'^* \triangleq \arg\min_{x',y'} \left\|\nabla W' - \nabla W\right\|^2 = \arg\min_{x',y'} \left\|\frac{\partial L(F(x',W),y')}{\partial W} - \nabla W\right\|^2$, they only need to update x' to realize the reconstruction of x with higher accuracy. These studies conclusively demonstrate the need for gradient privacy preservation in FL systems.

Homomorphic encryption

Homomorphic encryption (HE) is a cryptographic primitive that enables the performance of computations over encrypted data without disclosing the plaintext. A HE scheme is a tuple (*HE.KeyGen*(), *HE.Enc*(), *HE.Dec*(), *HE.Eval*()) of probabilistic polynomial time (PPT) algorithms

Table 1 Classification of Homomorphic Encryption

Type		Add	Mul	Scheme
PHE	Add	∞	0	Paillier (1999)
	Mul	0	∞	ElGamal (1985)
SHE		∞	$0 \leq l \leq L$	BGN Boneh et al. (2005) BFV Fan and Vercauteren (2012)
LHE		∞	$0 \leq l \leq L$	BGV Brakerski et al. (2014) CKKS Cheon et al. (2017, 2018, 2019)
FHE		∞	∞	FHEW Ducas and Micciancio (2015) TFHE Chillotti et al. (2020)

Gentry (2009). Since 1978, the RSA cryptosystem was born, and various schemes have been proposed Acar et al. (2018). As shown in Table 1, homomorphic encryption can be broadly divided into four types: partially homomorphic encryption (PHE), somewhat homomorphic encryption (SHE), leveled fully homomorphic encryption (LHE), and fully homomorphic encryption (FHE). This is based on whether a scheme allows addition and multiplication operations on ciphertext and its operation times. PHE supports either addition or multiplication operations under encryption, SHE and LHE support unlimited additions and finite multiplications, and FHE supports both without limitations. With the development of homomorphic encryption, the SHE concept is gradually replaced by LHE. Leveled FHE schemes based on the RLWE problem can be converted into fully homomorphic encryption schemes through bootstrapping operations.

Choosing an appropriate HE scheme for efficiently implementing a particular application is challenging for non-cryptography experts and lay users. Various FHE schemes possess unique benefits. For instance, BFV demonstrates proficiency in performing integer arithmetic, whereas TFHE exhibits rapidity in executing boolean algebra. BFV and BGV have demonstrated efficacy in supporting integer arithmetic, whereas FHEW and TFHE can implement boolean logic gates with remarkable speed. Jiang and Ju (2022) benchmarked major FHE schemes and concluded that the CKKS scheme is the preferred choice for target application that requires real or complex arithmetic operations. That is why the CKKS scheme has garnered significant attention as a practical tool for implementing privacy-preserving FL systems.

Related work

To defend against gradient leakage related attacks, various cryptographic tools have been employed to enhance the security of FL systems.

DP-based FedAvg

Differential Privacy (DP) Dwork (2006) is a mathematical technique used to protect individual privacy while allowing for practical data analysis by adding noise to the data in a carefully calibrated way. Zhu et al. (2020) combined federated learning with centralized differential privacy to discover frequent sequences, protecting user-level privacy to a certain extent. Truex et al. (2020) proposed a novel FL system with a formal privacy guarantee using local differential privacy (LDP). Wei et al. (2020) focused on information leakage in SGD based FL and proposed a novel framework based on the concept of global $(\epsilon, \delta) - \text{DP}$. They demonstrated that there is a tradeoff between the model performance and the allocation of privacy budget.

SMPC-based FedAvg

Secure Multi-Party Computation (SMPC) Cramer et al. (2015) is a series of cryptographic approaches for the secure evaluation of a public function on private data provided by multiple parties. Bonawitz et al. (2017) designed a secure aggregation protocol for FL that can tolerate client outages using blinding with random values, Shamir's Secret Sharing, and symmetric encryption. Kadhe et al. (2020) proposed a secure aggregation protocol based on the fast fourier transform and multi-secret sharing. SAFELearn Fereidooni et al. (2021) presented a generic framework for FL systems without a trusted third party.

HE-based FedAvg

Yang et al. (2020) designed and implemented a Paillier based secure FedAvg protocol and evaluated the computation and communication overhead after introducing Paillier. Zhang et al. (2020) utilized quantized gradients and batch coding to reduce high computational and communication costs caused by additive homomorphic encryption Paillier. Qiu et al. (2022) introduced a privacy-enhanced FedAvg based on the CKKS encryption scheme named PE-FedAvg. However, due to the utilisation of a simple logistic regression model, the achieved test accuracy on the MNIST dataset was limited to 82.07%. In our prior work Yao et al. (2023), we also constructed a variant of the FedAvg algorithm based on CKKS named BatchAgg. BatchAgg employs the Convolutional Neural Network (CNN) model and achieves a test accuracy of 98.40% on the MNIST dataset. While BatchAgg

demonstrates superior model accuracy, its evaluation has been limited to a certain dataset. Additionally, there is a dearth of comprehensive optimisation efforts for CKKS. Stan et al. (2022) investigated secure data aggregation in FL with DP, HE, and SMPC while considering different threat models. They demonstrate a delicate trade-off between gradients privacy preservation and model accuracy. Their evaluation results show that HE performs better for lower bandwidth usage than SMPC.

To sum up, each privacy protection technique has its pros and cons. Understanding and balancing these trade-offs, both theoretically and empirically, is a considerable challenge in realizing private federated learning systems.

~~Segmented CKKS~~

In this section, we will analyze the relationship of the security parameters in CKKS, evaluate the impact of multiplication depth and security level on computational performance, and introduce our proposed adaptive segmented encryption method for CKKS.

CKKS scheme

CKKS is a lattice-based FHE scheme that supports approximate arithmetics over real and complex numbers. Figure 1 provides a high-level overview of the CKKS scheme. In CKKS, the plaintext and ciphertext spaces include elements of the polynomial ring $R_q = \mathbb{Z}_q[x]/f(x)$, where q is an integer called the coefficient modulus and $f(x)$ is a polynomial known as the polynomial modulus. Elements of R_q are polynomials with integer coefficients bounded by q. The prevailing selection for the function $f(x)$ in scholarly works is $f(x) = x^N + 1$, where N (referred to as the ring dimension or polynomial modulus degree) is an integer that is a power of 2. The message $m \in \mathbb{C}^{N/2}$ is firstly encoded into a plaintext polynomial $p(X) \in R_q$ and then encrypted into two ciphertext polynomials $(c_0(X), c_1(X) \in R_q)$. Within the ciphertext domain, CKKS can perform homomorphic addition, multiplication, and rotation operations.

Multiplication depth

In CKKS, multiplication depth is the length of the longest chain of consecutive multiplications in a computation. For instance, $a_1 * a_2 * \ldots a_{n+1}$ has a multiplicative depth of n, whereas $a_1 * a_2 * \cdots * a_{n+1} + a_1 * a_2 * \cdots * a_{m+1}$ has a multiplicative depth of of $max(m, n)$. The maximum multiplication depth L for each ciphertext is specified during the key initialization phrase via the polynomial modulus degree and coefficient modulus:

- *polynomial modulus degree*: Polynomial modulus degree determines the maximum vector length that can be encrypted at one time. For CKKS, N/2 values can be encoded in a single ciphertext. The recommended values of N in the Homomorphic Encryption Standardization and Microsoft SEAL library Microsoft SEAL (2023) are 1024, 2048, 4096, 8192, 16384, and 32768.
- *coefficient modulus*: The coefficient modulus is a substantial integer formed by multiplying prime integers together. The prime numbers are organized within a

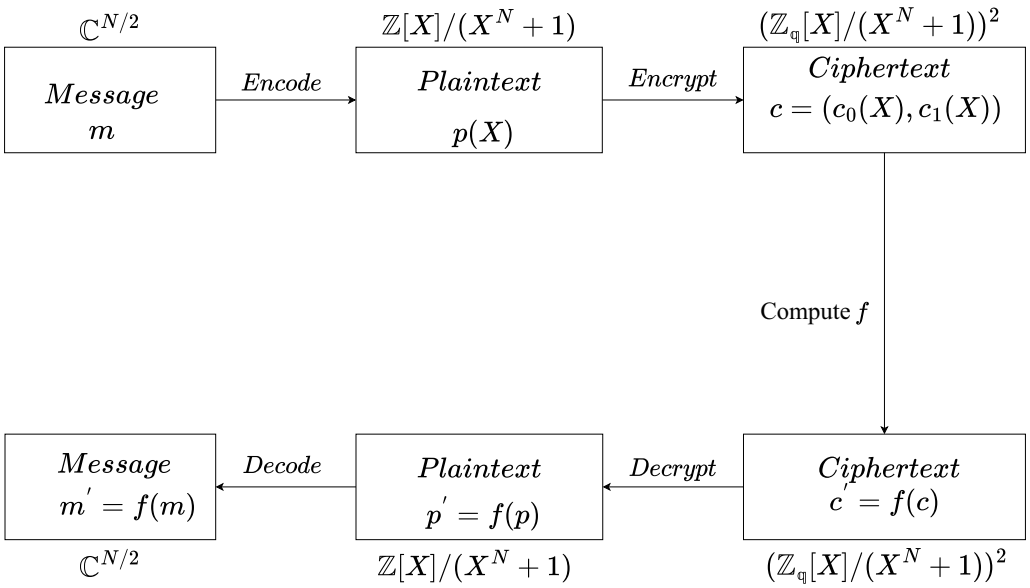


Fig. 1 Overview of the CKKS scheme

Table 2 Upper Bound of Coefficient Modulus

	Security Level		
	128-bit	**192-bit**	**256-bit**
N	Max q	Max q	Max q
1024	27	19	14
2048	54	37	29
4096	109	75	58
8192	218	152	118
16384	438	305	237
32768	881	611	476

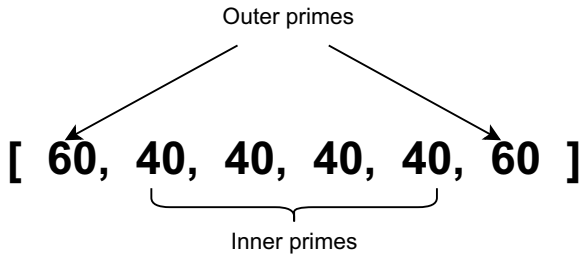


Fig. 2 Modulus chain used to determine multiplication depth

vector known as a modulus chain. The noise budget is determined by the coefficient modulus, with a larger coefficient modulus allowing for more homomorphic multiplication operations to be executed.

The upper bound of the coefficient modulus is determined by the polynomial module degree. As shown in Table 2, the homomorphic encryption standard gives the relationship between the polynomial modulus degree and the upper bound of the ciphertext coefficient modulus under different security levels.

As depicted in Fig. 2, the upper bound of the coefficient modulus is 438 for a polynomial modulus count of 16384 at 128-bit security level, and the modulus chain with multiplication depth of 4 can be set to [60, 40, 40, 40, 40, 60]. The array consists of two parts: the outer prime, which determines the precision of floating-point numbers, and the inner prime, the number of which determines the multiplication depth. In particular, when the polynomial modulus degree takes values of 1024 or 2048, the modulus chain contains only one value.

We first generated all possible modulus chains and then evaluated the effect of multiplication depth on the computational performance of encryption/decryption and homomorphic addition/multiplication under different polynomial modulus degrees with different multiplication depths. Figure 3 illustrates the evaluation results under a 128-bit security level.

Our evaluation results yield the following conclusions:

~~**Finding 1.** En~~cryption proves to be more time-consuming than decryption, and homomorphic multiplication incurs greater time overhead compared to homomorphic addition when using identical security parameters.

~~**Finding 2.**~~ In cases with equivalent security levels and polynomial modulus degrees, increasing the multiplication depth exacts a higher performance penalty. Conversely, opting for a lower polynomial modulus degree under the same multiplication depth results in reduced computational cost.

~~**Finding 3.**~~ When the multiplication depth is set to 0, CKKS exclusively supports homomorphic addition.

Security level

In cryptography, the security level is a measure of the strength of security that can be achieved by a cryptographic primitive, usually in bits. An encryption scheme achieving n-bit security implies that an attacker aiming to compromise the scheme must undertake a minimum of 2^n operations. We selectively display the evaluation results for a polynomial modulus degree of 32768. Figure 4 illustrates a comparative analysis of the time consumption for encryption, decryption, addition, and multiplication operations under security levels of 128-bit, 192-bit, and 256-bit.

In conjunction with Table 2, the following findings are derived:

Finding 4. As the security level increases, the upper bound on the coefficient modulus diminishes, and the supported multiplication depth decreases.

Finding 5. Holding the polynomial modulus degree and multiplication depth constant, the security level has no impact on computational performance.

Segmented CKKS

We introduce an adaptive segmented encryption algorithm tailored for the CKKS scheme by leveraging the insights derived from our aforementioned evaluation conclusions. This algorithm consists of the following sequential procedures:

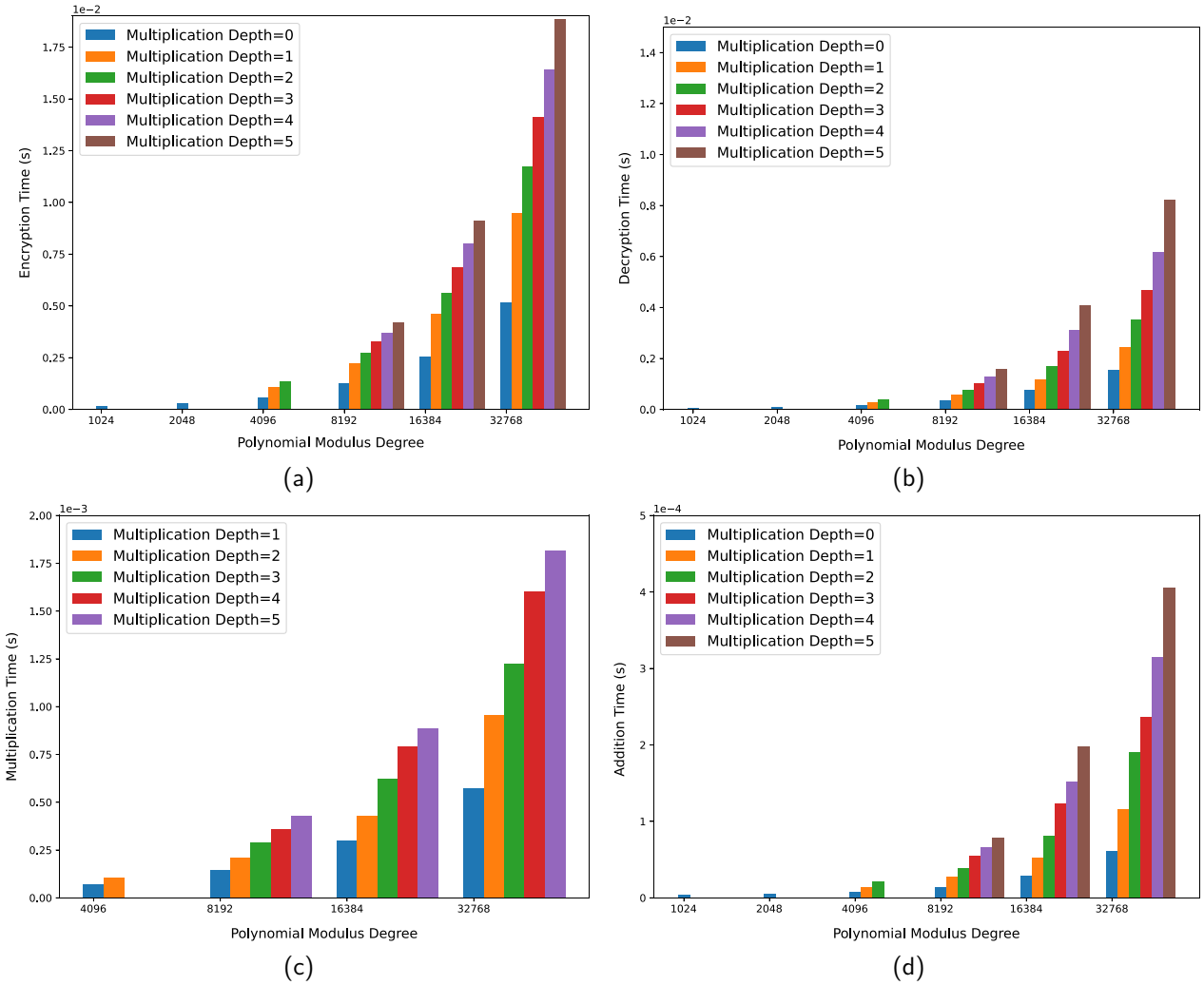


Fig. 3 Evaluation on computational performance under different multiplication depth. **a** Encryption. **b** Decryption. **c** Multiplication. **d** Addition

Algorithm 1 Depth-adaptive CKKS Key Generation Algorithm. *AutoKeyGen*(*d*, *S*, *ModDict*)

Require: multiplication depth: d,security level: s, module chain dict: $ModDict$
Ensure: public Key: P_k, private Key: S_k
$D_{arr} \rightarrow$ keys of ModDict$[s]$
$Max_d \rightarrow$ max of D_{arr}
if $d > Max_d$ or $d < 0$ **then**
    break
end if
$N_{arr} \rightarrow$ keys of ModDict$[s][d]$
$N \rightarrow min(N_{arr})$
Modulus Chain $MC \rightarrow ModDict[s][d][N]$
$P_k, S_k \rightarrow HE.KeyGen(N, MC)$
return N, P_k, S_k

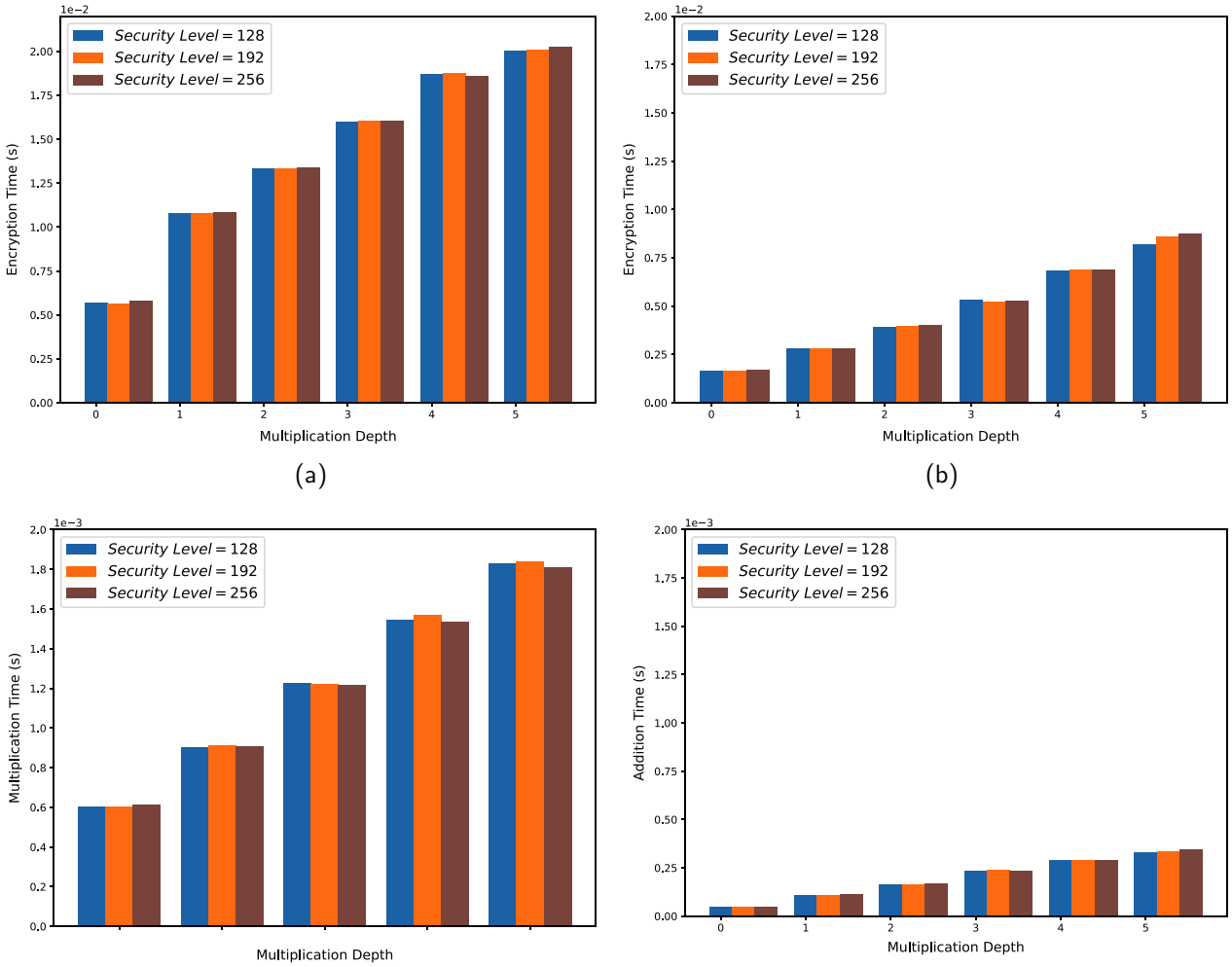


Fig. 4 Evaluation on computational performance under different security level. **a** Encryption. **b** Decryption. **c** Multiplication. **d** Addition

Step 1: Depth-adaptive Key Generation. The multiplication depth is specified at the key initialization phrase. According to **Finding 2**, we need to find the most suitable polynomial modulus degree that satisfies the security level and the multiplication depth. We first generate a modulus chain dictionary *ModDict*, in which the first-level key of the dictionary is the security level, the second-level key is the multiplication depth, the third-level key is the polynomial modulus degree, and the value is a list of modulus chains that satisfy the security level and multiplication depth requirements. Our depth-adaptive key generation algorithm is shown in Algorithm 1, which inputs the security level and multiplication depth and returns public/private key pair with an optimal polynomial modulus degree.

Algorithm 2 Segmented Encryption

Require: plain vector: V, public key: Pk,polynomial modulus degree: N
Ensure: segmented encryption array: ARR_{seg}

init ARR_{seg}
$l_v \leftarrow$ length of V
if $l_v < N/2$ **then**
    $ARR_{seg}.append(HE.Enc(V, Pk))$
else
    $l_g \leftarrow \lceil l_v/(N/2) \rceil$
    for $i = 0 \quad to \quad (l_g - 1)$ **do**
        $start_index \leftarrow (N/2) * i$
        $end_index \leftarrow (N/2) * (i + 1)$
        if $end_index > l_v$ **then**
            $end_index \leftarrow l_v$
        end if
        $V_g \leftarrow V[start_index : end_index]$
        $ARR_{seg}.append(HE.Enc(V_g, Pk))$
    end for
end if

return ARR_{seg}

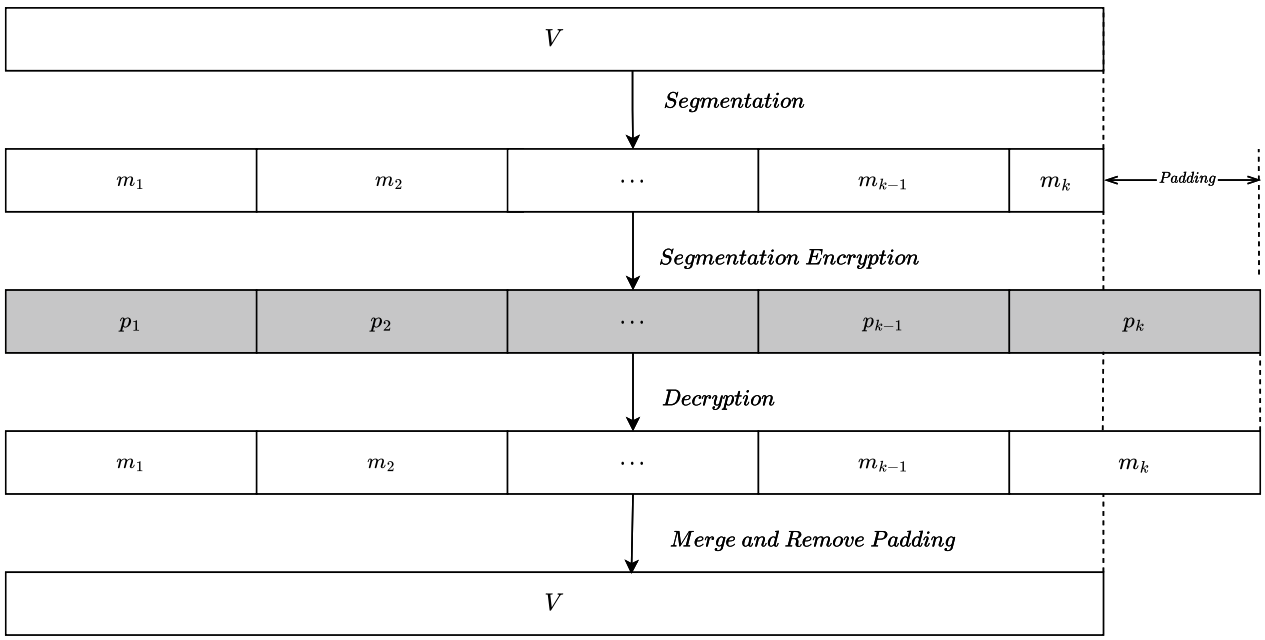


Fig. 5 Segmented encryption process of CKKS

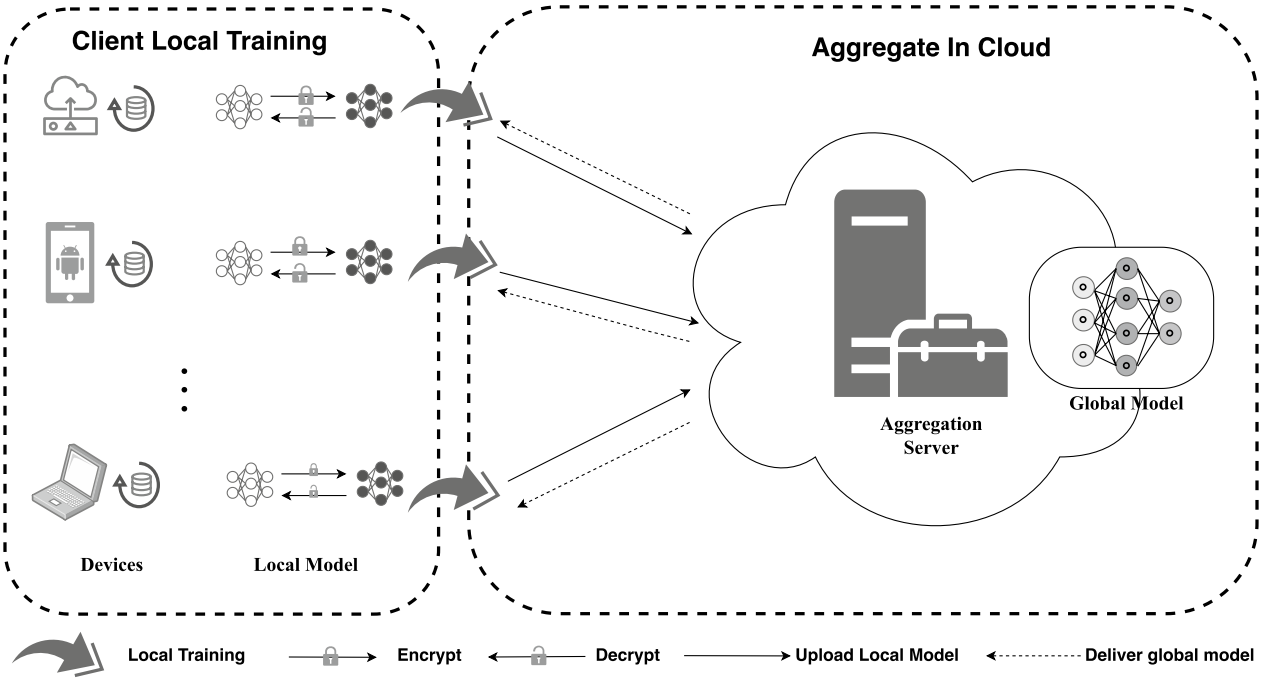


Fig. 6 Homomorphic encryption-based federated learning structure

Step 2: Segmented Encryption. Define V as the vector to be encrypted. If the length of V is less than half of the polynomial modulus degree, i.e., N/2, it can be encrypted directly. Otherwise, it is segmented. Algorithm 2 describes segmented encryption for CKKS in detail. V is first partitioned into k plaintext vectors $\{m_i\}_{i=1\cdots k}$ by length of N/2, where the length of the k-th vector $len(m_k) = len(V) \bmod (N/2)$. According to the CKKS encoding standard, if the length of the plaintext vector is less than N/2, it will be automatically padded to N/2 with 0. Therefore, there is a padding process. After segmented encryption, the plaintext vector V is encrypted into a ciphertext array $\{p_i\}_{i=1\cdots k}$.

Step 3: Segmented Decryption. Algorithm 3 shows the process of segmented decryption, where the input is a ciphertext array, and the output is a plaintext vector. Specifically, the standard decryption method is repeated to decrypt each ciphertext separately to obtain the plaintext array.

Algorithm 3 Segmented Decryption

Require: encryption array: ARR_{se}, private key: sk, vector length: l
Ensure: plain vector: V
1: init array: $varr$
2: **for** encv in ARR_{se} **do**
3: $varr.append(HE.Dec(encv, sk))$
4: **end for**
5: $V \leftarrow concatenate(varr)$
6: $V \leftarrow V[0:l]$
7: return V

Step 4: Decryption and Remove Padding. The last decrypted element contains padding elements. Therefore, it is necessary to merge the decrypted plaintext arrays and remove the padding value to obtain the correct decryption result.

As shown in Fig. 5, we show how a plain vector V is encrypted and decrypted using segmented encryption and decryption.

Proposed FedSHE scheme

In this section, we present our privacy-preserving and efficient federated learning framework, denoted as FedSHE, integrated with adaptive segmented CKKS homomorphic encryption.

~~System model~~

We consider a horizontal federated learning scenario implemented within a client–server architecture. Model parameters are frequently exchanged between multiple clients and an aggregation server. Given that gradients and model updates are mathematically hierarchical, our proposed system encrypts the model parameters using the segmented encryption CKKS described in Sect. 4 to defend against gradient leakage-related attacks.

Figure 6 illustrates our system structure, which consists of a key management center (KMC), an aggregation server (AS), and multiple clients. The KMC, a trusted organization, is responsible for conducting authentication procedures for the aggregation server and the clients. The KMC is also responsible for generating public/private key pairs and their subsequent distribution. The AS can choose clients during each iteration of Federated Learning (FL) and securely aggregate the trained model parameters from these clients. Clients engage in local training using their respective local datasets.

The primary objective of privacy-preserving FL is to strike a balance between privacy, utility, and efficiency. Specifically, for a secure and efficient HFL framework incorporating HE, the design goals include:

- **Privacy**: It must be ensured that the inference attack cannot be successfully executed even if malicious adversaries or a semi-honest server eavesdrops on the gradient transmissions.
- **Utility**: Despite the incorporation of HE, the global model must maintain high accuracy comparable to training in plaintext.
- **Efficiency**: The system should retain practical computational efficiency, with the computational cost and communication overhead resulting from the integration of homomorphic encryption minimized as much as possible.

~~Threat model and security analysis~~

The threat model is a prerequisite and basis for designing security protocols. According to the adversary's assault capabilities, the adversary in FL can be semi-honest or malicious. Under the semi-honest assumption, the adversary will comply with the computational protocol but will try to use intermediate information to reference more private information. In our proposed system model, we make the following assumptions:

- The clients are honest and do not abuse each other's data or collude with each other.
- The AS is semi-honest, also known as honest-but-curious, which means that the AS will not perform malicious operations such as decryption or reverse engineering on the gradients uploaded from the clients but potentially attempts to infer private training data from them.

We discuss how our algorithm resists gradient leakage attacks by ensuring the confidentiality of the gradients. Semantic security is a fundamental property of the HE scheme, meaning an adversary cannot recover plaintext from ciphertext within probabilistic polynomial-time (PPT). This characteristic is attributed to the incorporation of randomness in the encryption process, a fundamental aspect designed to enhance the security of these cryptographic systems. Specifically, during the encryption process, a random element is introduced, ensuring that the mapping of a plaintext to a ciphertext is nondeterministic. Consequently, even if the same plaintext is encrypted multiple times, each encryption operation generates a distinct ciphertext. This property, known as probabilistic encryption, is critical for preventing certain types of attacks, including those that attempt to derive the encryption key or plaintext by analyzing patterns in the ciphertexts. For instance, in the Paillier encryption scheme, each encryption of a plaintext involves the selection of a random number that plays a crucial role in the computation of the ciphertext. Similarly, the CKKS

scheme, while facilitating operations on encrypted data, also employs randomness in its encryption process to ensure that identical plaintexts encrypt to different ciphertexts on each occasion. This approach not only secures the encryption against a range of cryptographic attacks but also preserves the privacy of the data by obfuscating the relationship between repeated encryptions of the same data. The security of CKKS can be reduced to the ring learning with errors (RLWE) problem. Our segmented CKKS variant is an adaptation of the CKKS scheme, and consequently, it inherits the identical semantic security properties of the original CKKS scheme.

In our algorithm, the semi-honest aggregation server can not lanch privacy attack from the encrypted network weights sent by the clients. When the Paillier scheme is employed to encrypt model weights, the aggregation server is able to infer the number of model neurons based on the quantity of ciphertexts. Conversely, with the CKKS scheme, the server discerns the number of segments into which the weights are partitioned. However, neither method divulges the model's architecture, nor does it enable the aggregation server to conduct privacy attacks based on the ciphertexts. In the secure aggregation process, the aggregation server solely receives encrypted model weights and performs summation. Consequently, our algorithm effectively preserves the confidentiality of model weights, thus ensuring the privacy of data distributed among various clients.

FedAvg with segmented CKKS

In our proposed FedSHE scheme, the KMC selects the optimal CKKS security parameters according to the initialised global model, and then generates public-private key pairs. Next, the AS sends down the global model to the clients. Each client participating in the training process encrypts local model parameters trained with local data, using its private key to protect the trained local model parameters. After that, the clients transmit the encrypted local model parameters to the AS. The AS aggregates and averages the encrypted model parameters. Consequently, the proposed PPFL framework ensures data confidentiality between the clients and the AS. The training process consists of key distribution, global model initialization, and multiple aggregation rounds between the server and the clients. The detailed procedure of our proposed FedSHE scheme is described in the following steps.

Step 1: Global Model Initialization. A neural network model or other machine learning models can be used in our proposed FedSHE algorithm. The initialization of the global model W and the loss function $\mathcal{L}$ is consistent with the original FedAvg algorithm, described in Sec. 2. Before commencing model training, the AS distributes the global model W to designated clients.

Step 2: Homomorphic Key Generation and Distribution. The maximum vector length is derived from the global model M. Subsequently, the KMC employs our proposed depth-adaptive key generation algorithm, outlined in Algorithm. 1, to produce public and private keys of CKKS based on the maximum vector length. The AS only performs operations over the ciphertexts, so it only allows access to the public key. The clients need to decrypt encrypted model parameters with a private key for local training so they receive both. Before the KMC distributes the key pairs, it performs an authentication procedure for the clients and delivers the public-private key pairs to authenticated clients through secure channels.

Step 3: Client Local Training. The comprehensive pseudocode for an aggregation round between the clients and the AS is provided in Algorithm. 4. In the context of the initial round of model training, each client employs an optimization method, such as Stochastic Gradient Descent (SGD), to train local model on their local dataset. Conversely, for subsequent iterations, clients must utilize Algorithm. 6 for decrypting the encrypted model weights distributed by the AS before commencing the model training procedure. The training process is terminated after the client-side training process reaches predetermined epochs. Finally, the client utilizes Algorithm. 5 to encrypt the model weights and transmit the encrypted model parameters and the loss function to the server.

Algorithm 4 FedAvg with Segmented CKKS

Server executes:

initialize W_0
for each roud in $t = 1, 2, \ldots$ **do**
  $m \leftarrow \max(K \cdot \rho, 1)$
  $C_t \leftarrow$ (random set of m clients)
  for each client $c \in C_t$ **in parallel do**
    $\left[W_{t+1}^{c}\right] \leftarrow$ **ClientUpdate** $(c, [W_t^c])$
  end for
  average model weights:
  $[W_{t+1}] \leftarrow AvgModel\left(\left[W_{t+1}^{(c)}\right]_{c=1,\cdots,m}\right)$ ▷ Algorithm. 7
  deliver $[W_{t+1}]$ to all clients.
  if global model converges or satisfies termination condition **then**
    the server notifies all clients to stop local training
  end if
end for

ClientUpdate$(c, [W_t])$ // Run on client k
client c, $\forall c = 1, 2, ..., m$ in parallel do
decrypt model weights:
$W_t \leftarrow DecModel(Sk, [W_t])$ ▷ Algorithm. 6
$W_{1,1}^{c} = \bar{W}_t$
$\mathcal{B} \leftarrow$ (split $\mathcal{D}_c$ into batches of size B)
get local model parameters from last iteration: $W_{1,i}^{c} = W_{B,i-1}^{c}$
for each local epoch i from 1 to S **do**
  for batch $b \in \mathcal{B}$ **do**
    $W_{b+1}^{c} \leftarrow W_{b,i}^{c} - \eta \nabla \ell(W; b)$
  end for
end for
get $W_{t+1}^{(c)} = W_{B,S}^{(c)}$
encrypt model weights:
$\left[W_{t+1}^{(c)}\right] \leftarrow EncModel(Pk, W_{t+1}^{(c)})$ ▷ Algorithm. 5
upload $\left[W_{t+1}^{(c)}\right]$ and $\mathcal{L}_{t+1}^{c}$ to the aggregation server

Step 4: Model Aggregation. We design a secure aggregation algorithm, as illustrated in Algorithm. 7, for performing a weighted averaging of the encrypted model weights uploaded by clients. Thanks to the unique ciphertext computability function provided by homomorphic encryption, the server is able to directly perform accumulation operations on these encrypted weights. Note that the local model weights encrypted with the public key of the $i-th$ client at round t is represented as $[W_t^i]$. Upon completion of the aggregation process, the server computes their average, yielding the updated global model $[W_{avg}]$. Subsequently, the updated model is distributed to the clients for the commencement of a new round of model iteration training.

Algorithm 5 Encrypt Model Weights *EncModel*(*Pk*, *W*)

Require:
Public Pk, Model Parameters: W
Ensure:
encrypted model weights: $[W]$
for i, k in $enumerate(W.keys())$ **do**
    init vector: enc_vector
    neural network layer: $v \leftarrow W[k]$
    $enc_vector \leftarrow SegEnc(v, Pk)$ ▷ Algorithm. 2
    $[W][i] \leftarrow enc_vector$
end for
return $[W]$

Algorithm 6 Decrypt Model Weights *DecModel*(*Sk*, [*W*])

Require:
Private Key: Sk, Model Parameters: $[W]$
Ensure:
Plain Parameters W_{plain}
init W_{plain}
for k in $[W].keys()$ **do**
    $W_{plain}[k] \leftarrow SegDec([W][k])$ ▷ Algorithm. 3
end for
return W_{plain}

Algorithm 7 Aggregate Model Weights *AvgModel*($[W^c]_{c=1,\cdots,m}$)

Require: $[W^c]_{c=1,\cdots,m}$
Ensure: $[W_{avg}]$
for k *in* $[W^0].keys()$ **do**
    for $i = 1$ *to* m **do**
        $[W_{avg}][k] += w[i][k]$
    end for
end for
return $[W_{avg}]/m$

The training process between the AS and the clients proceeds through multiple rounds until the global model reaches convergence or satisfies the termination conditions.

Performance evaluation and analysis

Experimental setup

In this section, we conduct extensive simulations to evaluate our proposed FedSHE scheme. Our evaluation encompasses model utility, security level, computational efficiency, and communication cost. These simulation assessments are conducted on an Ubuntu 20.04 system equipped with an Intel(R) Core(TM) i5-12490F CPU running at 3.0GHz and 32GB of RAM, notably without GPU integration. Using the PyTorch framework, we establish a FL simulation environment for image classification and instantiated CNN models.

Datasets: We perform image classification experiments utilizing the MNIST and CIFAR-10 datasets. The MNIST dataset comprises 70,000 grayscale hand-written digit images, divided into a training set of 60,000 samples and a test set of 10,000 samples. The CIFAR-10 dataset consists of 60,000 color images, with 50,000 samples for training and 10,000 samples for testing. These two datasets' training samples are independently

and identically distributed to ten clients in our FL settings. The testing samples are used to assess the performance of the global model.

CNN Models: LeNet-5's fully connected and convolutional layers encompass more than 16,384 neurons, as detailed in Table 3. This characteristic renders it highly suitable for validating our segmented CKKS encryption algorithm. Furthermore, we utilize AlexNet to demonstrate the generalization of FedSHE.

Baselines: We employ three baselines for comparative analysis to demonstrate the advancement of our proposed segmented CKKS-based FedAvg: FedAvg, Paillier-based FedAvg Yang et al. (2020); He et al. (2022), and the original CKKS-based FedAvg Qiu et al. (2022); Yao et al. (2023). The enhancement of security comes at the cost of computational performance loss and increased communication overhead. After employing HE to enhance privacy, the introduction of homomorphic encryption results in additional computational and communication costs. Therefore, solutions with a lower additional computational cost than FedAvg indicate better performance.

Table 3 LeNet-5 model parameters

Network neural layer	LeNet-5 for MNIST Parameter amount	LeNet-5 for CIFAR-10 Parameter amount
conv1.weight	150	1200
conv1.bias	6	16
conv2.weight	2400	12800
conv2.bias	16	32
fc1.weight	48000	96000
fc1.bias	120	120
fc2.weight	10080	10080
fc2.bias	84	84
fc3.weight	840	840
fc3.bias	10	10
Total Parameters	61706	121182

HE parameters selection

The secure aggregation task in FedAvg involves only addition operations. Therefore, the additive HE scheme, Paillier, becomes a natural choice commonly used in privacy-preserving FL. Given the distinct security foundations of Paillier and CKKS, the primary consideration is the choice of parameter setting criteria. We performed experimental evaluations for both schemes at a consistent security level of 128 bits to enable a fair comparison.

For Paillier, the security level is determined solely by the key length. As of 2023, in accordance with NIST's recommendations, a minimum key size under 128 bits of security is 2048. For CKKS, according to **Finding 3**, configuring the CKKS multiplication depth to be 0 is sufficient to meet the homomorphic computation requirements. Additionally, in line with **Finding 4**, increasing the security level does not result in additional computational costs. Therefore, we also conducted evaluations under both 192-bit and 256-bit security levels, respectively.

The scaling factor, denoted as scale, is used to specify the precision during the conversion from floating-point numbers to fixed-point numbers. Table 4 presents the parameters configuration utilized in our experiments.

Simulation results of model performance

In accordance with prior studies Yang et al. (2020); He et al. (2022); Yao et al. (2023), HE-based FL, whether implemented with the Paillier or CKKS scheme, exhibits equivalent model accuracy to FL without privacy protection. We first verify the correctness of our proposed segmentation encryption algorithm. In Fig. 7a and b represent the global loss and test accuracy on the MNIST dataset, respectively, while subfig 7c and subfig 7d depict the global loss and test accuracy on the CIFAR-10 dataset. The legend FedAvg-Paillier represents FedAvg with the Paillier scheme, while FedAvg-SegCKKS indicates FedAvg combined with segmented CKKS. Their respective final test accuracies are as follows: on the MNIST

Table 4 CKKS parameters when multiplication depth is 0

Polynomial modulus degree	Scale	Modulus chain (128 bit-security)	Modulus chain (192 bit-security)	Modulus chain (256 bit-security)
1024	2^{20}	[27]	[19]	[14]
2048	2^{20}	[54]	[37]	[29]
4096	2^{20}	[30]	[30]	[30]
8192	2^{20}	[30]	[30]	[30]
16384	2^{30}	[60]	[60]	[60]
32768	2^{30}	[60]	[60]	[60]

dataset, 99.21%, 99.19%, and 99.22%; on the CIFAR-10 dataset, 65.90%, 65.85%, and 65.77%. The learning curves illustrate that the learning curves for the three training modes closely align, indicating that homomorphic encryption does not incur performance loss on FL models. This observation further substantiates the correctness of our proposed segmented CKKS encryption algorithm.

Table 5 Comparison of test accuracy on MNIST

	Test accuracy (%)
Paillier-FedAvg Yang et al. (2020)	96.01
Paillier-Plus-FedAvg He et al. (2022)	98.40
PE-FedAvg Qiu et al. (2022)	82.07
BatchAgg Yao et al. (2023)	98.40
FedSHE	99.22

We systematically assess and analyze the model accuracy in comparison to existing research endeavors. The evaluation results are presented in Table 5. These works can be categorized into two groups: one that employs CNN models, including works Yang et al. (2020), He et al. (2022), Yao et al. (2023). However, they do not provide specific details about the model architecture in their papers, and the reported testing accuracies are all slightly lower than ours. The other category involves the use of traditional machine learning models, with PE-FedAvg Qiu et al. (2022) being representative. Notably, adopting a relatively simple logistic regression model resulted in a testing accuracy of only 82.07% Our algorithm achieved the highest test accuracy, attributed to the adoption of the simple yet effective LeNet model and judicious hyperparameter configurations. Furthermore, it is noteworthy that these studies solely conducted evaluations on

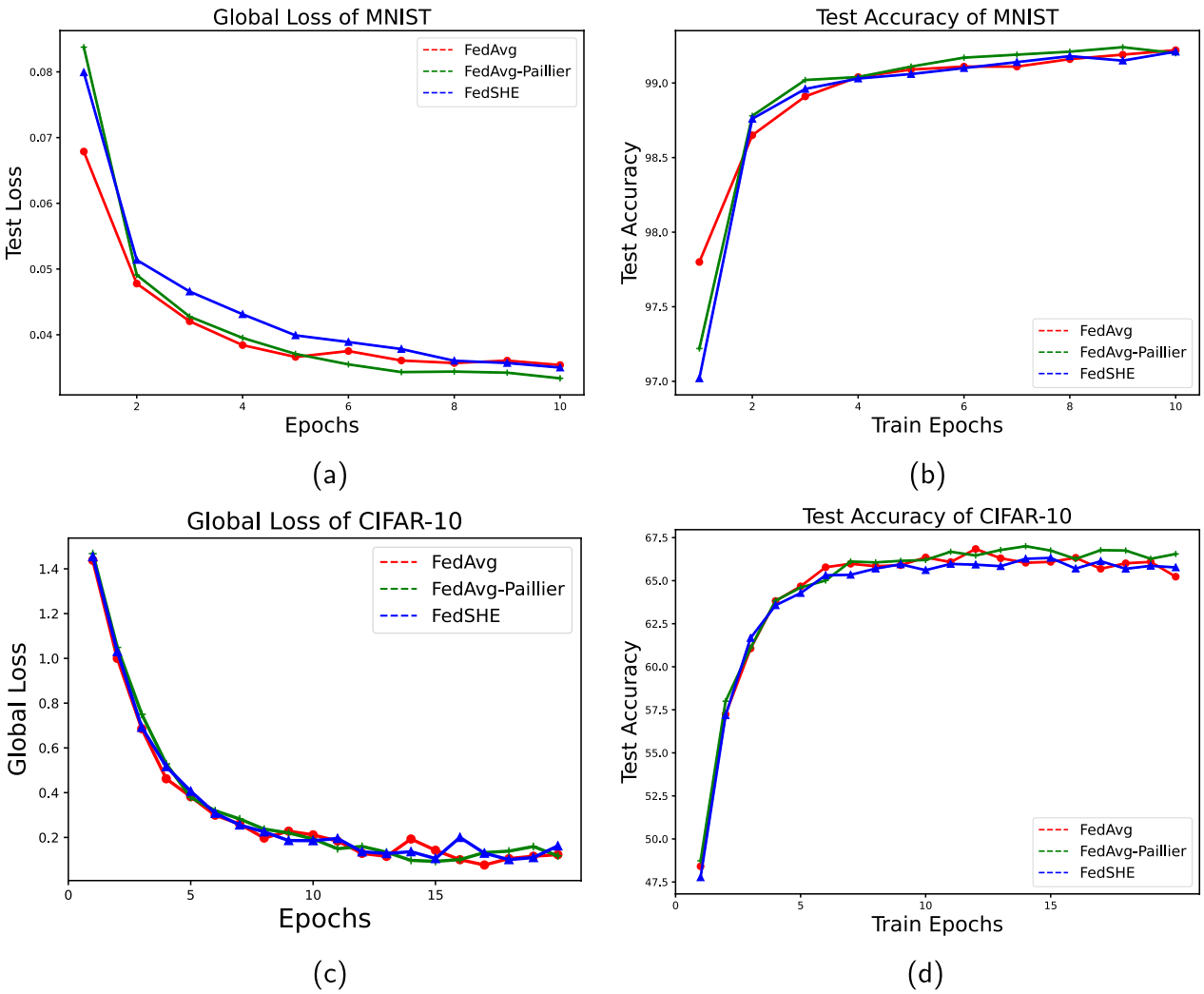


Fig. 7 Model performance **a** Global loss of MNIST. **b** Test accuracy of MNIST. **c** Global loss of CIFAR-10. **d** Test accuracy of CIFAR-10

the MNIST dataset without demonstrating the efficacy of the adopted models on other datasets.

Simulation results of computational efficiency

The computational time for each round of the vanilla FedAvg encompasses the following phases:

1. Global Model Initialization Time: The time consumption of model initialization in this phase is typically negligible as it occurs only once.
2. Client Local Training Time: The training time during this phase represents the client's training duration on its local dataset. It is influenced by factors such as the number of iterations, learning rate, and dataset size, denoted as T_{local}.
3. Model Parameter Transmission Time: This phase encompasses the time required for transmitting model parameters between the clients and the AS, denoted as T_{trans}.
4. Global Model Aggregation Time: The time consumption during this phase primarily involves the server's aggregation operations on the model parameters uploaded by the clients. Let T_{agg} represent the average time for global model aggregation.

In each training round, the total computational time T_{total} can be expressed as:

$$T_{\text{total}} = T_{\text{local}} + T_{\text{trans}} + T_{\text{agg}}$$

The adoption of HE introduces additional computational overhead, including T_{enc} and T_{dec}, representing the time overhead for traversing and segmentally encrypting/decrypting all layers of the model weights, respectively. The aggregation time for model parameters under ciphertext will certainly be higher than plaintext aggregation. The total time consumption per round after introducing homomorphic encryption is expressed as follows:

$$T_{\text{total}} = T_{\text{local}} + T_{\text{enc}} + T_{\text{trans}} + T_{\text{agg}} + T_{\text{dec}}$$

In the subsequent discussion, we use superscripts to denote the time consumption under different HE schemes. For instance, T_{total}^{ckks} represents the total time consumption under the CKKS scheme, $T_{enc}^{Paillier}$ denotes the encryption time under the Paillier scheme, and so forth. In a real environment, T_{trans} varies due to factors like network topology, bandwidth and communication protocols. In our simulated experiments, we do not include T_{trans} in the total time consumption. Table6 presents the computational performance evaluation results of HE-based FedAvg under different sizes. It is evident from the table that as the key length changes, T_{local} remains relatively consistent, with statistical errors within the range of $\pm 0.05s$.

(a) *Impact of Key Size*. Figure 8 illustrates the variation in computational time for T_{enc}, T_{agg}, and T_{dec} with key length changes. While a commonality exists in the sense that computational time increases as key length increases, there are also discernible differences. Specifically, in the Paillier scheme, with the increase in key length, $T_{\text{enc}}^{Paillier}$ and $T_{\text{dec}}^{Paillier}$ exhibit exponential growth. This is attributed to the security foundation of Paillier, rooted in decisional composite residuosity assumption, involving modular squaring operations. When the key length of Paillier reaches 2048 bits, $T_{\text{enc}}^{Paillier}$ for MNIST is 90 times T_{local}, and for the CIFAR-10 dataset, $T_{\text{enc}}^{Paillier}$ is 110 times T_{local}. In contrast, within the CKKS scheme, the growth of T_{enc}^{CKKS} and T_{enc}^{CKKS} is more

Table 6 Computational performance evaluation of HE-based FedAvg

Scheme	Key Size	MNIST				CIFAR-10			
		T_{local}	T_{enc}	T_{agg}	T_{dec}	T_{local}	T_{enc}	T_{agg}	T_{dec}
FedAvg	-	4.81	-	0.0007	-	6.22	-	0.0008	-
FedAvg with Paillier	128	4.79	1.43	3.01	0.37	6.21	2.85	6.13	0.69
	256	4.83	2.68	3.53	0.88	6.19	5.12	6.54	1.71
	512	4.77	10.49	4.95	3.26	6.12	19.80	9.78	6.18
	1024	4.82	61.58	9.58	18.79	6.25	116.24	18.29	35.52
	2048	4.76	430.88	24.98	121.95	6.17	818.73	47.46	229.99
FedAvg with Segmented CKKS	1024	4.78	0.023	0.004	0.007	6.31	0.045	0.006	0.012
	2048	4.81	0.025	0.004	0.007	6.25	0.048	0.006	0.013
	4096	4.79	0.028	0.004	0.008	6.18	0.053	0.006	0.016
	8192	4.80	0.033	0.005	0.010	6.14	0.059	0.007	0.019
	16384	4.77	0.046	0.007	0.014	6.27	0.67	0.009	0.021
	32768	4.83	0.067	0.013	0.023	6.23	0.092	0.013	0.027

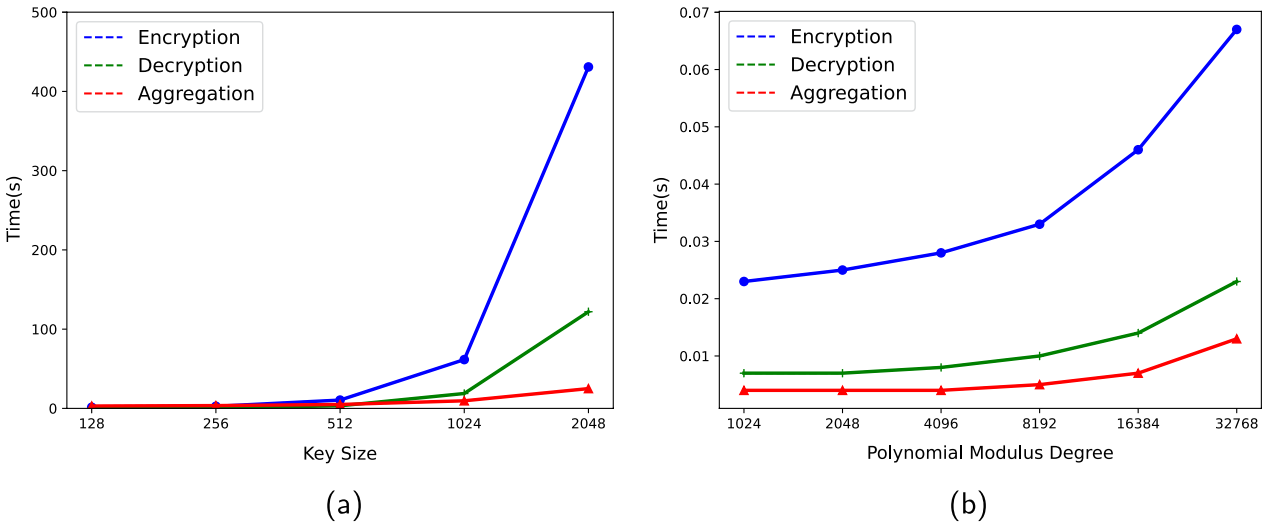


Fig. 8 Impact of key size on computational performance. **a** Paillier-based FedAvg on MNIST. **b** CKKS-based FedAvg on MNIST

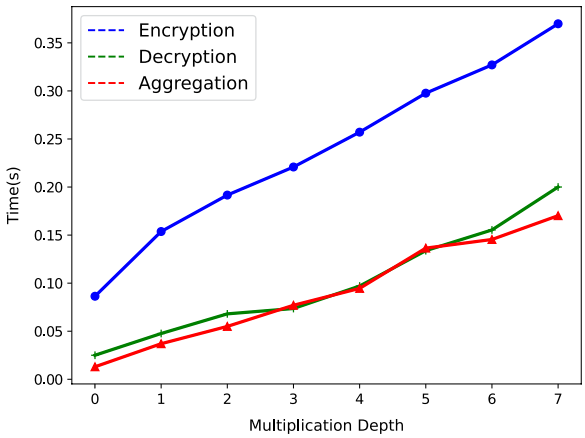


Fig. 9 Impact of CKKS multiplication depth on computational performance

gradual. The conclusion that can be drawn is that a minor polynomial modulus degree can shorten the encryption/decryption time, subsequently reducing the training time. This validates the effectiveness of our proposed segmented CKKS encryption algorithm.

(b) *Impact of Multiplication Depth*. We further validate the impact of multiplication depth on the time consumption for encryption, decryption, and ciphertext aggregation. Figure 9 depicts our evaluation results on the CIFAR-10 dataset, with the experimental premise being the polynomial modulus degree set to 32768 and the multiplication depth ranging from 0 to 7. What can be clearly seen in this figure is that, with the increase in multiplication depth, T_{enc}^{ckks}, T_{agg}^{ckks}, and T_{dec}^{ckks} also increase accordingly. Our evaluation results validate the conclusion of **Finding 2**, demonstrating the necessity of selecting an appropriate multiplication depth when employing CKKS.

Table 7 Comparison of overall training time

	T_{total}	
	MNIST	**CIFAR-10**
FedAvg	481.2	621.1
Paillier-FedAvg Yang et al. (2020)	58257.2	–
BatchAgg Yao et al. (2023)	533.2	–
FedSHE	484.6	631.7

(c) *Impact of Security Level*. We elevate the CKKS security level from 128-bit to 256-bit by adjusting the parameters outlined in Table 4. The evaluation results indicate that T_{enc}^{ckks}, T_{agg}^{ckks}, and T_{dec}^{ckks} remain unchanged, validating the conclusion from **Finding 5** that the security level does not impact computational performance. This implies we can enhance the security level without incurring additional computational cost.

Lastly, we conduct a comparative analysis of the overall computational efficiency with existing studies. Table 7 presents the total train time with 30 clients training ten global epochs. Although BatchAgg also employs the CKKS encryption scheme, the lack of consideration for parameter optimization and the improper setting of the multiplication depth result in a computational time that is 10% higher than ours. From Table 7, it is evident that our proposed segmented CKKS-based FedAvg exhibits lower computational latency compared to other

approaches. The total training time of Paillier-FedAvg is 121 times that of FedAvg. On the contrary, the additional training time of CKKS-based FedAvg compared with vanilla FedAvg on MNIST and CIFAR-10 only increases by 0.07% and 1.54%, respectively. The evaluation results demonstrate the practicality of our algorithm.

Simulation results of communication cost

Federated learning is frequently applied in edge devices, which face challenges related to bandwidth limitations, latency constraints, and resource-intensive communication protocols. Therefore, the communication cost stands as a significant challenge for HE-based FL systems in practical applications, particularly when FL is deployed in bandwidth-constrained scenarios. We conducted a further assessment of the impact of introducing HE on the communication cost of FL. In the subsequent discussion, the size of each double-precision floating-point number is considered to be 64 bits. We denote by *COMM* the communication cost for a single transmission of model parameters between the client and the server (Fig. 10).

For Paillier-based FedAvg, the Paillier encryption scheme encrypts neurons element by element, thus avoiding ciphertext redundancy. The size of each ciphertext after encrypting a plaintext number is equal to the length of the encryption key in bits. For instance, when the key size is 2048, each ciphertext is 2048 bits in size. The calculation method of communication cost only needs to multiply the number of elements by the size of a single ciphertext. The communication cost calculation formula under 128-bit security level is as follows:

$$COMM_{Paillier} = M \times 2048\ bit \tag{1}$$

For CKKS-based FedAvg, there is ciphertext redundancy, and the ciphertext size is closely related to the choice of the polynomial modulus degree. Let N represent the polynomial modulus degree, D represent the multiplication depth, and L represent a CNN with n layers, where L_i is the flattened vector of a specific layer. When the multiplication depth is 0, D is equal to 1, and so forth. Therefore, the formula for calculating the communication cost after adopting CKKS is as follows:

$$COMM_{CKKS} = \sum_{i=1}^{n} \left\lceil \frac{2 \times L_i}{N} \right\rceil \times N \times 2 \times D \times 64\ bit \tag{2}$$

We report padding elements under various polynomial modulus degrees, as presented in Table 8. It can be inferred that a more minor polynomial modulus degree leads to fewer padding elements. This aligns with the conclusions derived from the evaluations of computational efficiency, illustrating that the introduction of segmented CKKS encryption can also decrease communication costs.

Table 8 Padding elements under different polynomial modulus degree

Polynomial modulus degree	Padding elements	
	MNIST	CIFAR-10
1024	3318	3746
2048	6902	6818
4096	16118	13986
8192	32502	34466
16384	69366	67234
32768	134902	124578

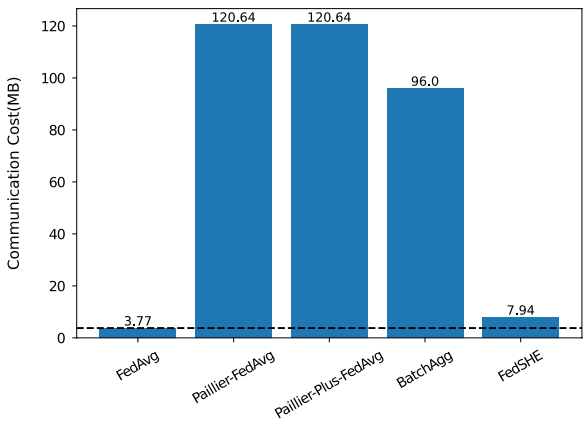


Fig. 10 Comparison of communication cost on MNIST

To sum up, we compared the communication cost with existing works. The baseline corresponds to plaintext-trained FedAvg, with a communication overhead of 3.76MB. Paillier-FedAvg and Paillier-Plus-FedAvg both employ Paillier encryption, resulting in a ciphertext expansion of 32 times. BatchAgg utilizes a polynomial modulus degree of 32768 and a multiplication depth of 3, leading to a communication cost of 96MB. Our algorithm incurs a communication overhead of only 6.6% compared to Paillier-based FedAvg and 8.3% compared to CKKS-based FedAvg. The evaluation results validate that our algorithm exhibits the lowest communication cost among the compared works.

Conclusions

In conclusion, this paper optimizes the tradeoff among privacy, utility, and efficiency to build reliable federated learning systems with homomorphic encryption.

Specifically, we present FedSHE, a privacy-preserving and efficient federated learning scheme with adaptive segmented CKKS homomorphic encryption. In the scheme, we propose a segmented encryption algorithm to break through the encryption length limitation of CKKS. Then, we evaluate the impact of polynomial modulus degree and multiplication depth on computational performance in the CKKS scheme and optimize parameter settings to enable computation while sacrificing as little as possible in performance. Finally, we implement FedSHE on top of the federated averaging (FedAvg) algorithm and evaluate public datasets. The experimental results demonstrate the correctness and effectiveness of our proposed method and indicate that FedSHE outperforms existing homomorphic encryption-based federated learning research efforts in terms of model accuracy, computational efficiency, communication cost, and security level. Our work also demonstrated a counterintuitive conclusion: fully homomorphic encryption is more efficient in federated learning than partially homomorphic encryption (Paillier), despite the simplicity and limited supported operations of the Paillier encryption process.

This paper exclusively focuses on utilizing homomorphic encryption to defend against data leakage attacks in horizontal federated learning. In future work, we plan to extend our research findings to broader scenarios, such as vertical federated learning, to further improve the efficiency of trustworthy federated learning.

Author contributions

The first author constructed the scheme and wrote the manuscript. All authors joined the discussion of the work, checked the validity of the scheme, read and approved the final manuscript.

Declarations

Competing interests

The authors declare that they have no competing interests.

Received: 16 January 2024 Accepted: 10 March 2024
Published online: 04 July 2024

References

Acar A, Aksu H, Uluagac AS, Conti M (2018) A survey on homomorphic encryption schemes: Theory and implementation. ACM Computing Surveys (Csur) 51(4):1–35

Albrecht M, Chase M, Chen H, Ding J, Goldwasser S, Gorbunov S, Halevi S, Hoffstein J, Laine K, Lauter K et al (2021) Homomorphic encryption standard. Protect Privacy Through Homomorphic Encrypt. 31–62

Aono Y, Hayashi T, Wang L, Moriai S et al (2017) Privacy-preserving deep learning via additively homomorphic encryption. IEEE Trans Inf Forensics Secur 13(5):1333–1345

Bonawitz K, Ivanov V, Kreuter B, Marcedone A, McMahan HB, Patel S, Ramage D, Segal A, Seth K (2017) Practical secure aggregation for privacy-preserving machine learning. In: Proceedings of the 2017 ACM SIGSAC conference on computer and communications security, pp 1175–1191

Boneh D, Goh E-J, Nissim K Evaluating 2-dnf formulas on ciphertexts. In: Theory of cryptography: second theory of cryptography conference, TCC 2005, Cambridge, MA, USA, February 10-12, 2005. Proceedings 2, pp. 325–341 (2005). Springer

Brakerski Z, Gentry C, Vaikuntanathan V (2014) (leveled) fully homomorphic encryption without bootstrapping. ACM Trans Comput Theory (TOCT) 6(3):1–36

Cheon JH, Han K, Kim A, Kim M, Song Y (2018) Bootstrapping for approximate homomorphic encryption. In: Advances in cryptology–EUROCRYPT 2018: 37th annual international conference on the theory and applications of cryptographic techniques, Tel Aviv, Israel, 2018 Proceedings, Part I 37, pp 360–384. Springer

Cheon JH, Han K, Kim A, Kim M, Song Y (2019) A full rns variant of approximate homomorphic encryption, pp. 347–368. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-030-10970-7_16

Cheon JH, Kim A, Kim M, Song Y (2017) Homomorphic encryption for arithmetic of approximate numbers. In: International conference on the theory and application of cryptology and information security, pp. 409–437. Springer

Chillotti I, Gama N, Georgieva M, Izabachène M (2020) Tfhe: fast fully homomorphic encryption over the torus. J Cryptol 33(1):34–91

Cramer R, Damgård IB, *et al.*: Secure Multiparty Computation. Cambridge University Press, ??? (2015)

Dimitrov DI, Balunovic M, Konstantinov N, Vechev M (2022) Data leakage in federated averaging. Trans Mach Learn Res

Ducas L, Micciancio D (2015) Fhew: bootstrapping homomorphic encryption in less than a second. In: Advances in cryptology–EUROCRYPT 2015: 34th annual international conference on the theory and applications of cryptographic techniques, Sofia, Bulgaria, Proceedings, Part I 34, pp. 617–640 (2015). Springer

Dwork C (2006) Differential privacy. In: International colloquium on automata, languages, and programming, pp 1–12. Springer

ElGamal T (1985) A public key cryptosystem and a signature scheme based on discrete logarithms. IEEE Trans Inf Theory 31(4):469–472

Fan J, Vercauteren F (2012) Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive

Fereidooni H, Marchal S, Miettinen M, Mirhoseini A, Möllering H, Nguyen TD, Rieger P, Sadeghi AR, Schneider T, Yalame H et al (2021) Safelearn: secure aggregation for private federated learning. In: 2021 IEEE security and privacy workshops (SPW), pp 56–62. IEEE

Geiping J, Bauermeister H, Dröge H, Moeller M (2020) Inverting gradients-how easy is it to break privacy in federated learning? Adv Neural Inf Process Syst 33:16937–16947

Gentry C A Fully Homomorphic Encryption Scheme. Stanford university, ??? (2009)

He C, Liu G, Guo S, Yang Y (2022) Privacy-preserving and low-latency federated learning in edge computing. IEEE Internet Things J 9(20):20149–20159

Jiang L, Ju L (2022) Fhebench: Benchmarking fully homomorphic encryption schemes. arXiv preprint arXiv:2203.00728

Kadhe S, Rajaraman N, Koyluoglu OO, Ramchandran K (2020) Fastsecagg: Scalable secure aggregation for privacy-preserving federated learning. arXiv preprint arXiv:2009.11248

Krizhevsky A, Sutskever I, Hinton GE (2017) Imagenet classification with deep convolutional neural networks. Commun ACM 60(6):84–90

LeCun Y, Bottou L, Bengio Y, Haffner P (1998) Gradient-based learning applied to document recognition. Proc IEEE 86(11):2278–2324

McMahan B, Moore E, Ramage D, Hampson S, Arcas BA. Communication-efficient learning of deep networks from decentralized data. Artif Intell Stat, 1273–1282 (2017)

Microsoft SEAL (release 4.1) (2023) https://github.com/Microsoft/SEAL. Microsoft Research, Redmond, WA

Paillier, P.: Public-key cryptosystems based on composite degree residuosity classes. In: International Conference on the Theory and Applications of Cryptographic Techniques, pp. 223–238 (1999). Springer

Qiu F, Yang H, Zhou L, Ma C, Fang L (2022) Privacy preserving federated learning using ckks homomorphic encryption. In: International conference on wireless algorithms, systems, and applications, pp. 427–440. Springer

Stan O, Thouvenot V, Boudguiga A, Kapusta K, Zuber M, Sirdey R (2022) A Secure Federated Learning: Analysis of Different Cryptographic Tools. In: Proceedings of the 19th International Conference on Security and

Cryptography—Vol. 1: SECRYPT, pp 669–674. SciTePress. https://doi.org/10.5220/0011322700003283 . INSTICC
Stripelis D, Saleem H, Ghai T, Dhinagar N, Gupta U, Anastasiou C, Ver Steeg G, Ravi S, Naveed M, Thompson PM, *et al.*: Secure neuroimaging analysis using federated learning with homomorphic encryption. In: 17th international symposium on medical information processing and analysis, 12088, 351–359 (2021). SPIE
Truex S, Liu L, Chow KH, Gursoy ME, Wei W (2020) Ldp-fed: Federated learning with local differential privacy. In: Proceedings of the third ACM international workshop on edge systems, analytics and networking, pp 61–66
Wei K, Li J, Ding M, Ma C, Yang HH, Farokhi F, Jin S, Quek TQ, Poor HV (2020) Federated learning with differential privacy: algorithms and performance analysis. IEEE Trans Inf Forensics Secur 15:3454–3469
Wei W, Liu L, Loper M, Chow KH, Gursoy ME, Truex S, Wu Y (2020) A framework for evaluating gradient leakage attacks in federated learning. arXiv preprint. arXiv:2004.10397
Yang Q, Liu Y, Chen T, Tong Y (2019) Federated machine learning: concept and applications. ACM Trans Intell Syst Technol (TIST) 10(2):1–19
Yang W, Liu B, Lu C, Yu N Privacy preserving on updated parameters in federated learning. In: Proceedings of the ACM turing celebration conference-China, pp. 27–31 (2020)
Yao P, Wang H, Zheng C, Yang J, Wang L (2023) Efficient federated learning aggregation protocol using approximate homomorphic encryption. In: 2023 26th international conference on computer supported cooperative work in design (CSCWD), pp 1884–1889. IEEE
Zhang C, Li S, Xia J, Wang W, Yan F, Liu Y (2020) BatchCrypt: Efficient homomorphic encryption for Cross-Silo federated learning. In: 2020 USENIX annual technical conference (USENIX ATC 20), pp 493–506
Zhao B, Mopuri KR, Bilen H (2020) idlg: Improved deep leakage from gradients. arXiv preprint arXiv:2001.02610
Zhu W, Kairouz P, McMahan B, Sun H, Li W (2020) Federated heavy hitters discovery with differential privacy. In: International Conference on Artificial Intelligence and Statistics, pp 3837–3847. PMLR
Zhu L, Liu Z, Han S (2019) Deep leakage from gradients. In: Annual conference on neural information processing systems (NeurIPS)

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.